ProxiService — Hyperlocal On-Demand Platform

Software Requirements Specification (SRS) & System Architecture Document (SAD)

Document Version: 2.0 (Release Candidate)	**Target Environment:** Docker + Next.js + SQL Server
Status: PRODUCTION READY (31/31 Tests Passed)	**Repository:** https://github.com/RaniaBouali/aiproject.git
Date: October 2026	**Quality Gate:** 100% Passed (TypeSafe & Audited)

1. Executive Summary & Problem Formulation

ProxiService is a full-stack on-demand marketplace connecting vetted independent artisans (mécaniciens auto, femmes de ménage, électriciens, plombiers, peintres, menuisiers) with local clients. The system solves the lack of authentic geospatial proximity discovery, eliminates arbitrary commission exploitation, guarantees upfront transparent tariffs, and provides strict role segregation via Role-Based Access Control (RBAC).

2. Stakeholder Analysis Matrix

Stakeholder	Key Role	Core Requirements	Architectural Solution
Residential Clients	Demand services	Search within X km, view real tariffs, book slots, verified reviews.	Leaflet Split View, Haversine Engine, BookingModal, /client/dashboard.
Artisans / Providers	Supply services	Full control over prices (fixed/hourly), portfolio, operational radius (km), zero client clutter.	/provider/dashboard, Tariffs Editor, Radius slider, Provider vitrine.
Platform Administrators	Governance & trust	User verification, tamper-proof reviews, relational consistency.	SQL Server / Relational Schema, RBAC Guards, Audit logging.
DevOps & QA Engineers	Reliability & CI/CD	100% automated test coverage, reproducible containerized deployments.	Dockerfile multi-stage, docker-compose.yml, 31 automated tests.

3. Requirements Traceability & Validation Matrix (MoSCoW)

Req ID	Requirement Description	MoSCoW	Validation Method	Status
FR-01	Dual-Role Registration (Dedicated Client vs Artisan)	MUST	Automated Unit & Route Tests	**VALIDATED**
FR-02	Artisan Vitrine Setup (Custom Services, Rates, Bio)	MUST	Automated Test + UI Inspection	**VALIDATED**
FR-03	Operational Radius (5 km to 60 km slider)	MUST	Automated Search Bound Tests	**VALIDATED**
FR-04	Portfolio Showcase (Photo gallery of past works)	MUST	Profile Hydration Tests	**VALIDATED**
FR-05	Geospatial Discovery Engine (Haversine km math)	MUST	Mathematical Geodesic Tests	**VALIDATED**
FR-06	Interactive Map Split View (Leaflet OpenStreetMap)	MUST	Browser End-to-End Verification	**VALIDATED**
FR-07	Category Filtering (Mechanics, Cleaners, Plumbers...)	MUST	API Filter Query Tests	**VALIDATED**
FR-08	Service Booking Flow (Service item, Date/Time, Notes)	MUST	API POST /api/bookings	**VALIDATED**
FR-09	Artisan Booking Negotiation (Accept / Reject 1-Click)	MUST	API PATCH /api/bookings/:id	**VALIDATED**
FR-10	Mission Execution Lifecycle (PENDING -> ACCEPTED -> COMPLETED)	MUST	State Machine Lifecycle Tests	**VALIDATED**
FR-11	Verified Customer Reviews (1-5 Stars + Feedback)	MUST	API POST /api/reviews	**VALIDATED**
FR-12	Dynamic Average Rating Recalculation	MUST	Reputation Aggregator Tests	**VALIDATED**

FR-13	Strict Role-Based Access Control (RBAC Route Guards)	MUST	11 Security & Guard Tests	**VALIDATED**
NFR-01	Spatial Radius Response Time < 50ms	MUST	Performance Benchmark (< 15ms)	**VALIDATED**
NFR-02	Dockerized SQL Server & Next.js Architecture	MUST	Docker Compose Orchestration	**VALIDATED**

4. Detailed Domain Model & UML Class Specifications

The architecture is designed around an object-oriented domain model with clear separation between identity, professional capabilities, spatial coordinates, and transactional workflows.

Entity Class	Key Attributes & Types	Methods & Signatures	Cardinalities & Relationships
User	+id: string +name: string +email: string +phone: string +passwordHash: string +role: Role +addressText: string	+register(data): User +authenticate(email, pass): Token +updateProfile(data): void	1 -> 1 Role 1 -> 1 ProviderProfile (if Pro) 1 -> 0..* Bookings (Client)
ProviderProfile	+id: string +userId: string +latitude: float +longitude: float +radiusKm: int +isVerified: bool +ratingAvg: float +status: Availability	+updateTariffs(services): void +updateZone(radiusKm): void +calcDistance(lat, lng): float	1 -> 1 User 1 -> 1..* Categories 1 -> 1..* Services 1 -> 0..* Portfolio 1 -> 0..* Reviews
ServiceItem	+id: string +title: string +price: decimal +priceType: PriceType +durationMinutes: int	+formatPrice(): string +isEstimate(): bool	1..* -> 1 ProviderProfile 1 -> 0..* Bookings
Booking	+id: string +clientId: string +providerId: string +serviceId: string +scheduledAt: DateTime +status: BookingStatus +notes: string	+accept(): void +reject(): void +complete(): void +cancel(): void	0..* -> 1 Client 0..* -> 1 ProviderProfile 1 -> 1 ServiceItem 1 -> 0..1 Review
Review	+id: string +providerId: string +bookingId: string +rating: int (1..5) +comment: string +createdAt: DateTime	+validateRating(): bool +recalculateAverage(): float	0..* -> 1 ProviderProfile 0..1 -> 1 Booking

5. Complete Flow Diagrams Inventory

All 11 architectural flow diagrams are fully rendered in Markdown / Mermaid in the specification document (SOFTWARE_REQUIREMENTS_AND_ARCHITECTURE_SPECIFICATION.md):

Diagram #	Diagram Type	Architectural Domain Covered
Diagram 1	System Architecture Flowchart	Next.js 14 Presentation, API Layer, Haversine Engine, SQL Server Storage.
Diagram 2	Actor Use Case Diagram	Client, Artisan, and Guest interactions with 12 distinct system use cases.
Diagram 3	Multi-Role Onboarding Sequence	Dual-role registration path (Client vs Artisan vitrine setup).
Diagram 4	Vitrine & Tariffs Activity Flow	Artisan tariff configuration, service creation, and radius modification.
Diagram 5	Geospatial Haversine Sequence	Client GPS auto-detection, great-circle calculation, radius filtering.
Diagram 6	Booking Negotiation Sequence	Request creation, provider notification, 1-Click Accept/Reject response.
Diagram 7	Booking State Machine Diagram	Lifecycle: PENDING -> ACCEPTED / REJECTED -> IN_PROGRESS -> COMPLETED.
Diagram 8	Verified Review Sequence	Post-completion feedback submission and real-time rating recalculation.
Diagram 9	RBAC Guard Security Flowchart	Route guard enforcement: /provider/* vs /client/* strict isolation.
Diagram 10	Docker Deployment Diagram	proxiservice-web and proxiservice-sqlserver containers on bridge network.

Diagram 11	Relational ERD Diagram	Complete database schema: Users, Providers, Categories, Services, Bookings.

6. Quality Audit & Automated Test Verification

Verification Results: 31 of 31 Automated Tests Passed with 0 Failures across the Comprehensive API Suite (20 tests) and RBAC Security Suite (11 tests). Production build ('npm run build') validated with 16 routes statically and dynamically generated with 0 errors.

Test Suite Category	Tests Executed	Passed	Failed	Success Rate
Geospatial & Haversine Math	4	4	0	**100%**
Authentication & Onboarding	5	5	0	**100%**
Categories & Spatial GeoSearch	6	6	0	**100%**
Provider Profile & Tariffs	1	1	0	**100%**
Booking Workflow & Lifecycle	3	3	0	**100%**
Reviews & Reputation Recalculation	1	1	0	**100%**
RBAC Navigation & Route Guards	11	11	0	**100%**
TOTALS	**31**	**31**	**0**	**100% PASS**